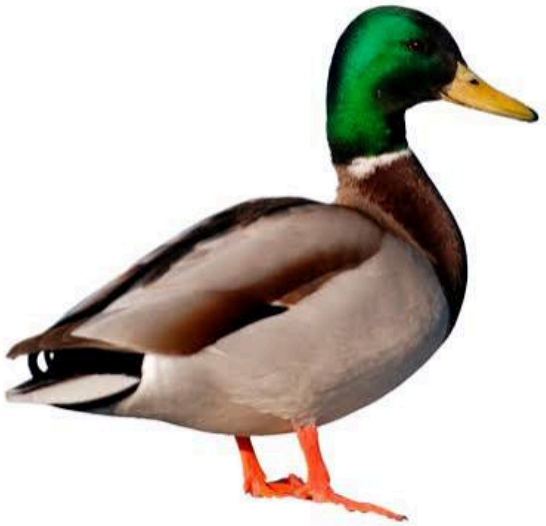
Ciência de Dados Aplicada ao Direito I

Aula 21

Quem ganha as ações de canabidiol



Prof. Bruno da Cunha de Oliveira - 2026/2

Reposição da aula de 28/10 e um atendimento

Vou marcar duas coisas na semana de 09/11: a reposição da aula de quarta, 28/10, com presença coletiva, e um horário de atendimento. Os horários seguem quatro regras do Insper:

#	Regra
1	sem conflito com aula, atendimento, monitoria ou atividade extra da turma
2	sem conflito com outra reposição, mesmo que ainda em análise
3	pelo menos 45 minutos de almoço entre 11h e 15h
4	pelo menos 15 minutos entre uma atividade e outra, para a locomoção

Horário da reposição

Opção	Dia	Horário	Duração
A	seg 09/11	12h às 13h30	1h30
B	seg 09/11	12h30 às 14h	1h30
C	seg 09/11	16h30 às 18h30	2 horas
D	qui 12/11	9h45 às 11h45	2 horas
E	sex 13/11	7h30 às 9h30	2 horas

Horário do atendimento

Opção	Dia	Horário	Duração
A	seg 09/11	12h às 13h30	1h30
B	seg 09/11	12h30 às 14h	1h30
C	seg 09/11	16h30 às 18h30	2 horas
D	qui 12/11	9h45 às 11h45	2 horas
E	sex 13/11	7h30 às 9h30	2 horas

Preparação para a Avaliação Final

O quê	Como
Lista de funções	15 exercícios, do fácil ao acima da prova. Quatro vêm marcados como recomendados, e a gente corrige esses quatro junto no atendimento de quinta, 05/11, às 12h30
Lista de classes	no mesmo formato, com os recomendados corrigidos no atendimento de 12/11
Simulado	quarta, 25/11: impresso, com menos tempo que a prova, e corrigido em sala questão a questão

Basta procurar o padrão

Ler o padrão que o assistente escreveu

Pedaço	O que diz
`Requerid[oa]s?:`	Requerido, Requerida, Requeridos ou Requeridas, e os dois-pontos
`\s*`	espaços ou tabulação, quantos houver
`([^\n]+)`	tudo até a quebra de linha; os parênteses marcam o que guardar

Não é para escrever: é para conferir se o que o assistente entendeu é o que vocês pediram.

31 ficaram vazias

```
269
0       Fazenda Pública do Estado de São Paulo e outro
1    UNIMED DE GUARULHOS COOPERATIVA DE TRABALHO ME...
2    Fazenda Pública do Estado de São Paulo e INSTI...
3                                                  NaN
4                              UNIMED NACIONAL e outro
Name: requerido, dtype: str
```

`str.extract` guarda o trecho que corresponde ao padrão; onde não há, fica `NaN`, o vazio. As 31 não usam `Requerido:` (`Impetrado`, `Executado`, sem cabeçalho).

Dá para reduzir as 31?

Peçam ao assistente uma segunda rodada: "31 ficaram vazias; olhe os textos sem valor e amplie o padrão para os outros rótulos do polo passivo". Rodem e contem de novo.

Quantas sobram? E por que a regra sempre vai deixar alguma?

288 de 300, e as 12 que sobram

RESPOSTA

```
padrao = r"(?:Requerid|Impetrad|Executad|Reclamad)[oa]s?:
\s*([^\n]+)"
decisoes["requerido"]
= decisoes["texto"].str.extract(padrao)
print(decisoes["requerido"].notna().sum())
```

Impetrado, Executado e Reclamado entram; sobram 12 sem cabeçalho nenhum.
O padrão cresce a cada rodada, e sempre há um texto que ele não descreve.

Quem é mais acionado?

Com a coluna `requerido` pronta, `value_counts` conta quantas vezes cada valor aparece:

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
decisoes["requerido"] = decisoes["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
print(decisoes["requerido"].value_counts().head(6))
```

Olhem a lista que sai. Ela responde à pergunta 2 do cliente?

A mesma Fazenda, escrita de três jeitos

RESPOSTA

```
requerido
Fazenda Pública do Estado de São Paulo             66
Fazenda Pública do Estado de São Paulo e outro     28
Sul América Companhia de Seguro Saúde               9
PREFEITURA MUNICIPAL DE JABOTICABAL e outro          8
PREFEITURA MUNICIPAL DE SÃO PAULO                    4
PREFEITURA MUNICIPAL DE LIMEIRA e outro              4
Name: count, dtype: int64
```

Contar ainda não é responder

O que a lista mostra	O que falta
`Fazenda Pública do Estado de São Paulo` e a mesma `... e outro`	juntar: é o mesmo réu
`PREFEITURA MUNICIPAL DE X` em caixa alta	padronizar a grafia
`e outro`, `e outros`	separar o segundo réu

Padronizar é a aula de limpar uma base. Hoje a resposta ao cliente vem com essa ressalva escrita.

Público ou privado

Como dividiriam?

Temos a coluna `requerido`, com Fazenda Pública, prefeituras, Sul América, Unimed, Amil, Bradesco Saúde. Como vocês dividiriam esses valores entre público e privado, numa regra que o programa aplique às 300 linhas?

Uma regra por palavras

RESPOSTA

```
publico = []
for requerido in decisoes["requerido"]:
    nome = str(requerido).upper()
    if ("FAZENDA" in nome or "ESTADO" in nome
            or "MUNIC" in nome or "PREFEITURA" in nome):
        publico.append(True)
    else:
        publico.append(False)
```

O que a regra diz

```
decisoes["publico"] = publico
print(decisoes["publico"].value_counts())
```

Um `for` sobre a coluna, um `if` com `in` por palavra, e a lista `publico` vira coluna nova. `str()` porque o vazio não é texto; `.upper()` para "Prefeitura" e "PREFEITURA" contarem igual; `MUNIC` pega Município e Municipal.

187 públicos, e o resto não é tudo privado

```
publico
True     187
False    113
Name: count, dtype: int64
```

Entre as 300: 187 com ente público; 82 sem nenhuma das palavras, que a regra chama de privado; e as 31 sem `Requerido:`, que viraram "nan" e caíram no `False` junto com elas.

Primeira resposta ao cliente, com ressalva

Nesta base, mais de dois terços das ações são contra o Estado ou um município.

A ressalva é o termo de busca: "canabidiol" puxa quem pede ao SUS. Contra um plano de saúde, talvez se escreva de outro jeito, e essas sentenças ficaram fora da coleta.

Quem ganha

Insper

Procurar "procedente"

```
procedentes = 0
improcedentes = 0
for texto in decisoes["texto"]:
    if "procedente" in texto:
        procedentes = procedentes + 1
    if "improcedente" in texto:
        improcedentes = improcedentes + 1
print(procedentes)
print(improcedentes)
```

Sai 165 e 68. O que está errado nessa conta?

165 inclui os 68

RESPOSTA

```
165
68
```

"improcedente" contém "procedente": toda sentença que diz "improcedente" entra também na conta de "procedente". Os 165 incluem os 68. E "PROCEDENTE" em caixa alta escapa das duas contas.

Procurar uma palavra solta não responde "quem ganhou". O padrão precisa dizer o que vem antes e depois dela.

A melhor regex que o assistente conseguiu

Pedi: "a melhor expressão regular possível para dizer se a sentença julgou procedente, parcialmente procedente, improcedente ou extinguiu sem mérito". Voltou isto, em quatro grupos. É para olhar, não para decifrar:

```
PADRAO = re.compile(r"""
    (?P<parcial>
        julgo\s+(?:\w+\s+){0,3}?
        (?:parcialmente\s+procedentes?
          |procedentes?\s+em\s+parte)
      | parcial\s+procedência)
```

... o grupo improcedente ...

```
  | (?P<improcedente>julgo\s+(?:\w+\s+){0,3}?improcedentes?
      | denego\s+a\s+(?:segurança|ordem)
      | rejeito\s+o\s+pedido
      | nego\s+provimento)
```

... o grupo procedente ...

```
    | (?P<procedente>julgo\s+(?:\w+\s+){0,3}?procedentes?
          (?!\s+em\s+parte)
        | concedo\s+a\s+(?:segurança|ordem)
        | acolho\s+o\s+pedido)
```

... e o grupo extinto

```
    | (?P<extinto>julgo\s+extint[oa]s?\b[^.]{0,160}?
          sem\s+(?:resolução|julgamento|exame|análise)
          \s+d[oe]\s+mérito
        | extingo\s+o\s+(?:processo|feito)[^.]{0,160}?
          sem\s+(?:resolução|julgamento)\s+d[oe]\s+mérito
        | art(?:igo|\.)?\s*485
        | homologo\s+(?:a\s+desistência|o\s+acordo
                      |a\s+transação))
""", re.IGNORECASE | re.VERBOSE)
```

A função que aplica o padrão

```
def resultado(texto):
    achados = list(PADRAO.finditer(texto[-3000:]))
    if not achados:
        return "?"
    return achados[-1].lastgroup

decisoes["resultado"] = decisoes["texto"].apply(resultado)
print(decisoes["resultado"].value_counts())
```

Procura só no fim do texto, onde fica o dispositivo, e fica com a última frase que casou. Sem frase nenhuma, "?".

40 que ela não alcança

RESPOSTA

```
resultado
procedente      130
improcedente     88
?                40
parcial          27
extinto          15
Name: count, dtype: int64
```

Quatro grupos de frases, e 40 de 300 sem classe.

Complexa, e ainda assim incompleta

O que a regex faz	O que escapa
procura nos últimos 3.000 caracteres	dispositivo mais acima
quatro grupos de frases: julgo, concedo, denego, extingo	juiz que escreve de outro jeito
fica com a última frase que casou	sentença que julga dois pedidos

A regra acerta onde o texto segue o padrão que eu imaginei. Para saber quantas estão certas, alguém lê uma amostra.

Fase 2: banca, demonstração e nota

Insper

A banca da fase 2

Quem avalia

Pessoas que trabalham com tecnologia. Os nomes vão por e-mail quando estiverem confirmados.

A banca da fase 2

Quem avalia

Pessoas que trabalham com tecnologia. Os nomes vão por e-mail quando estiverem confirmados.

O que costuma pesar

Em bancas de tecnologia: se o produto funciona de verdade; se aguenta erro; se o código é legível; se o grupo sabe explicar o que o assistente faz.

A demonstração

Quando	O quê
Até 17/11	o endereço do repositório
Em 18/11	o pitch, como na fase 1: slides e demonstração do jeito que o grupo preferir

Vale o que o produto faz

Vender o produto pode: o pitch continua sendo um pitch.

Dizer que ele faz uma coisa que não faz zera a nota da fase 2. O que vocês quiserem que a banca avalie tem de estar implementado no repositório.

Investir num grupo

Ponto	Regra
Quem	cada integrante da Temis, se quiser, em um grupo
Como	um contrato com o grupo, que fica comigo, e um fato relevante, que a turma vê
Até	o início da aula de 17/11, por e-mail, separado da entrega do grupo
Resultado	de +1,0 a −1,0, pela colocação do grupo investido no ranking da fase 2

A tier list com dez grupos

Faixa	S	A	B	C	D	F
Vagas	1	2	2	2	2	1
Pontos	15	12	9	6	3	1

As mesmas faixas e os mesmos pontos da fase 1. São dez vagas para dez grupos: nenhuma faixa leva um grupo a mais.

A nota da fase 2

Posição	Nota
1ª e 2ª	1,5
3ª a 6ª	1,4; 1,3; 1,2; 1,1
7ª a 10ª	de 0,1 a 1,0, pelo repositório

Outra maneira: utilizando Inteligência Artificial

Insper

Quem mais poderia preencher as três colunas?

Em vez de descrever o padrão do texto, dá para pedir a alguém que leia cada sentença e responda: quem é o réu, se é público ou privado, qual foi o resultado.

Ler 300 à mão leva um dia. Quem mais lê texto e responde em formato fixo?

Um modelo de linguagem

RESPOSTA

Um modelo de linguagem lê a sentença e preenche o formulário. Chamado de dentro do programa, ele faz isso para cada linha da tabela. Hoje vamos só usar; como ele funciona por dentro e o que custa ficam para as próximas aulas.

Para que o modelo de linguagem faça isso, precisamos de três coisas: uma chave de acesso a um serviço que roda o modelo, um formulário com as perguntas que queremos fazer, e a biblioteca que liga os dois à tabela.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

Hoje a chave vai colada no programa, e isso não é seguro: como guardá-la fora do código é assunto de uma aula adiante. Não mandem a chave a ninguém, nem para mim.

Ler o README de uma biblioteca

A biblioteca é o dataframeit, no GitHub. O README de uma biblioteca costuma ter o que basta para a primeira chamada: como instalar, um exemplo mínimo e a lista de argumentos.

```
uv add "dataframeit[groq]"
```

Vamos ler juntos o exemplo do README e adaptá-lo às nossas três perguntas.

O formulário

```
from typing import Literal
from pydantic import BaseModel
class Formulario(BaseModel):
    requerido: str
    polo_passivo: Literal["público", "privado"]
    resultado: Literal["procedente",
                       "parcialmente procedente",
                       "improcedente",
                       "extinto sem julgamento do mérito"]
```

O que cada linha da classe diz

Um campo por linha: nome, dois-pontos e tipo. `Literal` fecha a lista de valores que o modelo pode devolver; `str` é texto livre. A classe herda de `BaseModel` o trabalho de conferir cada resposta.

Uma chamada para dez sentenças

```
from dataframeit import dataframeit
dez = decisoes.head(10)
tabela = dataframeit(dez, Formulario,
    "Leia a sentença e preencha os campos: {texto}",
    provider="groq", text_column="texto",
    api_key="gsk_...")
print(tabela[["requerido", "polo_passivo", "resultado"]])
```

`head(10)` separa as dez primeiras das 300, para a chamada ser curta. Uma linha de cada vez: o texto vai a um servidor da Groq, o modelo responde, a classe confere. O caderno grava `tabela` em `dez-decisoes-modelo.csv`.

Os argumentos do dataframeit

Argumento	O que é
`dez`	as dez primeiras sentenças, uma por linha
`Formulario`	o formulário, que confere cada resposta
o texto com `{texto}`	o pedido ao modelo; cada sentença entra no lugar de `{texto}`
`provider="groq"`	o serviço que roda o modelo
`text_column="texto"`	a coluna em que está o texto
`api_key="gsk_..."`	a sua chave, colada hoje no programa

Rodem nas dez

Com a chave no lugar de `gsk_...`, rodem a célula. Leva alguns segundos por sentença.

Deu certo quando `tabela` tem dez linhas e três colunas novas, cada uma com um valor que a classe aceitou. Comparem com o que a regex deu para as mesmas dez.

Os dois caminhos lado a lado

Guardar antes de comparar: `to_csv`

```
dez = decisoes.head(10)
dez["requerido_regex"] = dez["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
dez.to_csv("dez-decisoes-regex.csv", index=False)
```

`to_csv` grava a tabela num arquivo; `index=False` deixa de fora a numeração das linhas. O caderno gravou `tabela`, a do modelo, do mesmo jeito. Com os dois arquivos salvos, a comparação não depende de rodar tudo de novo.

As mesmas dez, regex contra modelo

```
import pandas as pd

regex = pd.read_csv("dez-decisoes-regex.csv")
modelo = pd.read_csv("dez-decisoes-modelo.csv")

regex["requerido_modelo"] = modelo["requerido"]
print(regex[["requerido_regex", "requerido_modelo"]])
```

`read_csv` relê os dois arquivos. Onde as duas colunas diferem, quem acertou?

Quando usar cada um?

Nas dez sentenças, comparem os resultados e discutam em dupla:

#	Pergunta
1	Em que tarefa a regex basta, e por quê?
2	Em que tarefa o modelo é melhor, e o que ele custa?
3	Como vocês saberiam quantas respostas do modelo estão certas?

Regra para o que tem forma, modelo para o que tem sentido

RESPOSTA

	Expressão regular	Modelo de linguagem
acerta	em texto de forma fixa	em texto variado
custa	testar a regra	uma chamada por linha
erra	em silêncio	com confiança
se confere	lendo o vazio	lendo uma amostra

O que ainda vamos aprender

Etapa	A pergunta que ela responde
definir a amostra	que termo, classe, período e tribunal, e por quê
o que coletar e o que não	que dado de pessoa fica de fora, e com que base
coletar	raspar e guardar, com a ficha do que foi feito
limpar	a mesma Fazenda com um nome só; vazios e duplicados
descrever	contar, agrupar e cruzar: público contra privado, quem ganha em cada grupo
visualizar	o gráfico que o cliente lê sem a tabela

Na próxima aula

A regra contra o modelo de linguagem nas mesmas sentenças, até saber quando usar cada um. Depois, um cliente novo, uma grande companhia aérea, e as perguntas de pesquisa que valem a pena fazer por ela.